

Pioneer.jl Flanking-Window Features

Feature reference

2026-06-24

1 Motivation: wide-isolation-window DIA

In data-independent acquisition with *wide* quadrupole isolation windows, a single MS2 window co-isolates many precursors, so a confident identification needs evidence that the signal is real and not bleed-through from a co-isolated neighbor or chromatographic interference. The **flanking-window features** provide this by contrasting two regions of the *same* isolation window:

- the **core**: the contiguous run of MS2 cycles, centered on the precursor’s apex scan, where the precursor passes the search threshold (it is PSM-backed by construction — one MS2 scan per cycle per window);
- the **flank**: scans of the same isolation window in the cycles immediately *flanking* the core, where the precursor has *no* passing PSM.

A genuine precursor concentrates MS1 and fragment signal in the core with co-eluting fragments; interference smears into the flank or shows fragments that do not track the precursor.

2 Where they are computed (two pipeline stages)

1. **MainSearch** (`SearchMethods/MainSearch/scoring.jl`) — after best-per-precursor selection, `_mainsearch_flanking_core_bounds!` (:811) finds the core window, and the per-precursor loop (:1045--1097) writes the intermediate “core” columns into the fold Arrow files: `flanking_core_scan_min`, `flanking_core_scan_max`, `n_contiguous_scans`, `flanking_core_ms1_m0_signal`, `flanking_core_frag_signal`, plus the smoothed apex columns `frag1..8_smoothed_intensity`.
2. **PrecursorScoringSearch** (`PrecursorScoringSearch.jl`:199 → `add_wide_window_features_to_fold_files` in `wide_window_features.jl`) — reads the core columns, gathers the flank scans, extracts MS1 + fragment intensities from the raw spectra, computes the 11 flank feature values in `_wide_flank_feature_values` (:58), and writes them back. The two `*_signal` core columns are intermediate and dropped afterward (:893).

The final list `FLANKING_WINDOW_FEATURES` (`model_config.jl`:49) feeds the second-pass `ScoringSearch` LightGBM.

3 Defining the core, and what is raw vs smoothed (MainSearch)

`_mainsearch_flanking_core_bounds!` sorts a precursor’s scored rows by (`cycle`, `scan`), finds the apex (selected) scan, and walks outward while each neighbor both passes and is in the immediately adjacent cycle (± 1):

```
while lo>1 && pass_mask[prev] && cycle[prev]==cycle[cur]-1; lo-=1; end # left
while hi<len && pass_mask[next] && cycle[next]==cycle[cur]+1; hi+=1; end # right
```

It returns the core scan range `scan_min/scan_max`, `n_contiguous_scans`, and the apex/left/right boundary rows.

Core *signals* are summed from RAW intensities. Over the core scans (`scoring.jl:1082--1093`):

$$\text{core_ms1_m0_signal} = \sum_{\text{core scans}} \text{ms1_m0}, \quad \text{core_frag_signal} = \sum_{\text{core scans}} \sum_{r=1}^8 \text{frag}_r^{\text{raw}}.$$

Here $\text{frag}_r^{\text{raw}}$ are the matched per-scan intensities (`MAINSEARCH_FRAGMENT_INTENSITY_COLUMNS`); **no smoothing is applied to the sum.**

The smoothed values are a separate, per-fragment APEX height. `_mainsearch_radius1_smooth_fragme (:917)` computes, for each of the 8 fragments, a radius-1 weighted average over the apex scan and its two adjacent core scans:

$$\text{frag}_r^{\text{smoothed}} = \frac{2a_r + l_r + r_r}{\text{denom}},$$

where a_r, l_r, r_r are the raw apex/left/right intensities and `denom` counts the present neighbors. This is *not* summed — it is a denoised *peak height*. A single-scan raw apex is spiky; the 3-point weighted average is a stabler estimate of the true peak. These `frag*_smoothed_intensity` values are used (i) directly as LightGBM features and (ii) as the core apex in the flanking `frag_apex_gt2x` feature below. *So smoothing benefits the apex/peak-height comparison, not the sum.*

4 Gathering the flank and extracting signal (PrecursorScoringSearch)

- `_wide_window_index_spectra (:286)` indexes every MS2 scan by its isolation window key (`center_mz, isolation_width`) — one scan per cycle.
- `_wide_group_flank_window_scans_from_core! (:420)` collects the same-window scans in cycles flanking the core (outside `[scan_min, scan_max]`, within `WIDE_WINDOW_CYCLE_MARGIN=3` cycles). No flank scans $\Rightarrow$ zero/default feature values.
- `_wide_top_fragment_idxes / _wide_group_fragment_windows` pick the top-8 library fragments and m/z windows; scan-centric extraction (`_wide_fill_scan_centric_{ms1_m0, fragments}!`, batched at 50000 groups) fills the *raw* flank vectors `flank_ms1_m0` and `flank_fragments` ($\# \text{flank scans} \times 8$).

5 The features (`_wide_flank_feature_values`, line 58)

Let c_{m0}, c_f be the core MS1-m0 / fragment signals (raw sums); a_r the smoothed core apex of fragment r ; $\mathbf{m}$ the raw flank MS1-m0 vector (one per flank scan); F the raw flank fragment matrix (scan $\times$ fragment). Floor all intensities at 0; let $s_j = \sum_r F_{j,r}$, and $\rho(\cdot, \cdot)$ be Pearson correlation (`_wide_window_pcor`; 0 for < 2 points or zero variance).

<code>flanking_ms1_m0_candidate_fraction</code>	[Float32]
$\frac{c_{m0}}{c_{m0} + \sum_j \mathbf{m}_j}$ — fraction of the precursor’s MS1 monoisotopic (m0) signal sitting in the core vs. spread into flank scans. $\rightarrow 1$ for a clean precursor.	
<code>flanking_frag_candidate_fraction</code>	[Float32]
$\frac{c_f}{c_f + \sum_j s_j}$ — the same core-vs-flank fraction for summed fragment signal.	
<code>flanking_ms1_frag_sum_corr</code>	[Float32]
$\rho(\mathbf{m}, \mathbf{s})$ across flank scans — do the MS1 precursor and the fragment sum co-vary off-apex? (co-elution sanity check in the flank).	
<code>flanking_frag_corr_mean</code>	[Float32]
mean of all pairwise $\rho(F_{:,a}, F_{:,b})$ over flank scans, among fragments with signal — average fragment-fragment co-elution in the flank.	
<code>flanking_n_correlated_fragments</code>	[UInt8]
count of fragments whose leave-one-out correlation $\rho(F_{:,r}, \mathbf{s} - F_{:,r}) > 0.7$ (WIDE_WINDOW_CORR_THRESHOLD) — how many fragments track the rest.	
<code>flanking_n_correlated_fragments_bitvec_rank</code>	[UInt16]
<code>_bitvec_pattern_rank</code> of the 8-bit mask of <i>which</i> fragments exceeded 0.7 — a learned categorical encoding of the correlated-fragment <i>pattern</i> , not just the count.	
<code>flanking_frag_corr_strength</code>	[Float32]
$\sum_r \max(\min(\rho_r, 1), 0)$ over the leave-one-out correlations ρ_r — continuous total correlation strength across the 8 fragments.	
<code>flanking_frag_corr_effective_n</code>	[Float32]
$\frac{(\sum_r \rho_r^+)^2}{\sum_r (\rho_r^+)^2}$ (participation ratio of the positive leave-one-out correlations) — the effective number of co-eluting fragments.	
<code>flanking_frag_corr_best_m0</code>	[Float32]
$\rho(F_{:,b,\mathbf{m}})$ where b is the fragment with the highest mean pairwise correlation (the “anchor”) — does the best fragment track the MS1 precursor across the flank?	
<code>flanking_signal_support</code>	[Float32]
$\frac{\#\{j : \mathbf{m}_j > 0 \text{ or } s_j > 0\}}{\#\text{flank scans}}$ — fraction of flank scans carrying any signal.	

`frag_apex_gt2x_flank_bitvec_rank` [UInt16]
`_bitvec_pattern_rank` of the 8-bit mask of fragments whose smoothed *core apex* $a_r \geq 2\times$ their flank average $\bar{F}_{:,r}$ — which fragments genuinely peak at the apex rather than being flat/interference. (This is the one feature that uses the smoothed apex.)

`n_contiguous_scans` [UInt16]
(from the core bounds, not `_wide_flank_feature_values`) the number of contiguous cycles in the core run — how many cycles the precursor persisted.

The 11 values from `_wide_flank_feature_values` plus `n_contiguous_scans` make up the 12-entry `FLANKING_WINDOW_FEATURES`.

6 Helper functions

`_mainsearch_flanking_core_bounds!` (`scoring.jl:811`) [MainSearch]
Find the contiguous-cycle core window around the apex; return `scan_min/max`, `n_scans`, and the apex/left/right boundary rows.

`_mainsearch_radius1_smooth_fragments!` (`scoring.jl:917`) [MainSearch]
Radius-1 weighted apex smoothing $(2a + l + r)/\text{denom}$ producing `frag*smoothed_intensity`.

`_wide_window_pcor` (:5) [wide_window_features.jl]
Single-pass Float32 Pearson correlation; 0 if < 2 points or zero variance.

`_wide_candidate_signal_fraction` (:32) [wide_window_features.jl]
 $\text{core}/(\text{core} + \text{flank})$, guarding divide-by-zero.

`_wide_flank_feature_values` (:58) [wide_window_features.jl]
Compute the 11 flank features from the core scalars/apex and the raw flank arrays.

`_wide_window_index_spectra` (:286) [wide_window_features.jl]
Index scans by isolation window (`center_mz`, `isolation_width`).

`_wide_group_flank_window_scans_from_core!` (:420) [wide_window_features.jl]
Collect same-window scans in cycles flanking the core (margin = 3 cycles).

`_wide_fill_scan_centric_{ms1_m0,fragments}!` (:578/:601) [wide_window_features.jl]
Read raw-spectra MS1 m0 and fragment intensities at each flank scan (scan-centric, batched).

`_bitvec_pattern_rank` [shared]
Map an 8-bit fragment pattern mask to a learned rank (file-specific table).

7 Constants

WIDE_WINDOW_CYCLE_MARGIN= 3 (flank reaches up to 3 cycles either side); WIDE_WINDOW_CORR_THRESHOLD= 0.7 (correlated-fragment cutoff); FLANKING_SCAN_CENTRIC_BATCH_SIZE= 50000 (groups per scan-extraction batch).

8 The two LGBM feature sets

Pioneer trains two gradient-boosted classifiers with different feature sets. The flanking features belong only to the second one.

8.1 MainSearch per-file LGBM — PRESCORE_FEATURES (49 features)

Trained per file on *all* (scan, precursor) PSMs, *before* best-per-precursor selection (`features.jl:144`); used to pick the best scan per precursor and for the global prescore aggregation.

```
# Core PSM / sequence (13)
fitted_manhattan_distance, irt_error, poisson, err_norm, total_ions,
missed_cleavage, y_count, weight, gof, Mox, spectrum_peak_count,
sequence_length, fitted_hellinger
# Cross-precursor at-scan (2)
weight_ratio_at_scan, weight_rank_at_scan
# MS1 (11)
ms1_m0_mass_err_ppm, ms1_weight_apex_to_m0_apex_irt, ms1_m0_intensity,
ms1_m1_intensity, ms1_m1_to_m0_ratio, ms1_m1_to_m0_pred,
ms1_isotope_dotp_m0_m1_m2, ms1_m0_m1_m2_window_fraction,
ms1_ms2_explained_delta, ms1_m0_m1_m2_window_fraction_pc,
ms1_ms2_explained_delta_pc
# Top-8 fragment trace intensities, RAW (8)
frag1_int, frag2_int, frag3_int, frag4_int,
frag5_int, frag6_int, frag7_int, frag8_int
# Fragment-chromatogram correlations (6)
frag_apex_dispersion_irt, n_correlated_fragments,
n_correlated_fragments_bitvec_rank, frag_corr_strength,
frag_corr_effective_n, frag_corr_best_m0
# Misc (4)
top3_ms2_mass_error_mean, delta_frame_peak_center, log_by_ratio_m0, n_scans
# Backported from ADVANCED (5)
prec_mz, irt_fwhm, smoothness, log2_intensity_explained, longest_y
```

8.2 ScoringSearch (Pass-1) LGBM — ADVANCED_FEATURE_SET (73 features)

Trained *after* best-per-precursor selection, cross-run (`model_config.jl:85`; both the MBR and no-MBR paths use it, `score_psms.jl:270/362`). It mirrors PRESCORE_FEATURES but (i) substitutes the raw `frag*_int` with the smoothed apex `frag*_smoothed_intensity` (the best PSM is now known), and (ii) adds the fragment-detection, cross-window, and flanking-window families that require the selected best scan.

```
# Core PSM / sequence (13) -- same as PRESCORE
fitted_manhattan_distance, irt_error, poisson, err_norm, total_ions,
missed_cleavage, y_count, weight, gof, Mox, spectrum_peak_count,
sequence_length, fitted_hellinger
# Cross-precursor at-scan (2)
weight_ratio_at_scan, weight_rank_at_scan
```

```

# MS1 (11) -- same as PRESCORE
ms1_m0_mass_err_ppm, ms1_weight_apex_to_m0_apex_irt, ms1_m0_intensity,
ms1_m1_intensity, ms1_m1_to_m0_ratio, ms1_m1_to_m0_pred,
ms1_isotope_dotp_m0_m1_m2, ms1_m0_m1_m2_window_fraction,
ms1_ms2_explained_delta, ms1_m0_m1_m2_window_fraction_pc,
ms1_ms2_explained_delta_pc
# Top-8 fragment apex, SMOOTHED (8) -- substitutes frag*_int
frag1_smoothed_intensity .. frag8_smoothed_intensity
smoothed_2d_shadow_hellinger # (1)
# Fragment-chromatogram correlations (6) -- same as PRESCORE
frag_apex_dispersion_irt, n_correlated_fragments,
n_correlated_fragments_bitvec_rank, frag_corr_strength,
frag_corr_effective_n, frag_corr_best_m0
# Fragment-detection union/intersection (4)
n_fragments_detected_union, n_fragments_detected_intersection,
n_fragments_detected_union_bitvec_rank, n_fragments_detected_intersection_bitvec_rank
# Cross-window / transmission (4)
precursor_fraction_transmitted, n_scans_other_windows,
other_window_weight_corr, other_window_apex_delta_irt
# Flanking-window features (12) -- FLANKING_WINDOW_FEATURES (this document)
flanking_ms1_m0_candidate_fraction, flanking_frag_candidate_fraction,
flanking_ms1_frag_sum_corr, flanking_frag_corr_mean,
flanking_n_correlated_fragments, flanking_n_correlated_fragments_bitvec_rank,
flanking_frag_corr_strength, flanking_frag_corr_effective_n,
flanking_frag_corr_best_m0, flanking_signal_support,
n_contiguous_scans, frag_apex_gt2x_flank_bitvec_rank
# MS1-peak / window-precursor counts (3)
ms1_m0_peak_frag_intensity_fraction, ms1_m0_peak_n_precursors,
scan_prec_mz_n_precursors
# Misc (3)
top3_ms2_mass_error_mean, delta_frame_peak_center, log_by_ratio_m0
# RT / shape / explained (6)
n_scans, prec_mz, irt_fwhm, smoothness, log2_intensity_explained, longest_y

```

(A separate second-pass MBR-FTR model adds match-between-runs features when MBR is enabled; those are out of scope here.)

9 Proposed refactor: gather the core in PrecursorScoringSearch

The core-bounds, core-signal, and radius-1 smoothing are currently computed in *MainSearch* (`scoring.jl`, `_mainsearch_flanking_core_bounds!`), then carried forward as columns; the flank is computed later in *PrecursorScoringSearch* (PSS). Two facts argue for moving the core computation into PSS, next to the flank:

1. **Only ScoringSearch consumes these features.** None of the core/flanking/smoothed columns appear in `PRESCORE_FEATURES` (the *MainSearch* per-file LGBM). They exist *solely* for the *ScoringSearch* `ADVANCED_FEATURE_SET`. So the *MainSearch* best-PSM selection gains nothing from computing them.
2. **The core is raw-matched, not deconvolved.** The core fragment signal and the radius-1 smoothed apex are built from `MAINSEARCH_FRAGMENT_INTENSITY_COLUMNS` (`frag1_int..frag8_int`) — the *matched observed* intensities, the same kind of value the flank extracts raw from the spectrum. (The deconvolved `fitted_frag*_int` feed only `smoothed_2d_shadow_hellinger`.) So a PSS-side raw extraction of the core scans would be of the *same nature* as the current core — no deconvolution is lost.

Why it lives in MainSearch today. Data locality, not quality. The core spans several cycles, but the fold Arrow files retain only one best row per precursor; the per-core-scan matched intensities exist only *during* MainSearch (before the reduction to best-per-precursor), so the multi-scan sum and smoothing are done there and the results carried forward.

The refactor. In PSS, extract the core scans from the raw spectra exactly as the flank is extracted (same isolation window, adjacent cycles — incremental to the flank read), then smooth those values there. Net effect: work shifts MainSearch→PSS (roughly compute-neutral), the carried `flanking_core_*` columns are dropped, all flanking logic is consolidated in one place, and — a real win — the candidate-fraction and `frag_apex_gt2x` comparisons become apples-to-apples (core and flank both from the same PSS extraction, instead of today’s matched-core vs. raw-flank scale mix).

Caveat. The search’s matched intensities are not bit-identical to PSS’s `_wide_fragment_peak_intensity` extraction (different peak-pick / tolerance / isotope summation), so this is a deliberate *rebaseline* — validate ID/PG counts on the standard datasets, like any results-changing change.

10 Consumption

The features are appended to the per-fold second-pass PSM Arrow files and feed the ScoringSearch second-pass LightGBM (`FLANKING_WINDOW_FEATURES`). They are most informative on wide-window instruments (e.g. Sciex SWATH), where co-isolation is heaviest, and near-constant on narrow-window data.